

Fault-Onset Labeling Bounds Reported Accuracy in the Tennessee Eastman Benchmark: A Leakage-Controlled Study of Six Classical Classifiers

Abstract

Fault diagnosis in the Tennessee Eastman Process (TEP) is a standard benchmark for data-driven process monitoring, yet published results are difficult to compare. We present a leakage-controlled benchmark of six classical classifiers: logistic regression, decision tree, random forest, histogram-based gradient boosting, linear support vector machine and XGBoost. Data were partitioned by complete simulation runs, standardization was fitted on training data only, hyperparameters were selected on a separate validation partition, variability was measured over five seeds, and the frozen model was evaluated once on the complete official test partition of 10.08 million observations. XGBoost ranked first in repeated validation (macro-F1 0.7683 ± 0.0017) and reached a test macro-F1 of 0.7200, accuracy of 0.6754 and MCC of 0.6621 under the labels distributed with the dataset.

These figures are bounded by the benchmark itself. Each fault is injected 160 samples into every 960-sample test run, so literal labeling caps the recall of every fault class at 0.8333; faults 6 and 7 attain 0.8333 and 0.8332, the ceiling itself. Rescoring the same frozen model over the post-onset segment, without refitting, raises macro-F1 to 0.8054, accuracy to 0.7949 and MCC to 0.7852, and the apparent validation-to-test generalization gap of +0.0483 becomes -0.0125 under a symmetric comparison. Across seven classifiers the relative accuracy gain lies between 0.142 and 0.150 against a structural bound of 0.1587, so the effect is a property of the labeling convention rather than of any estimator. Normal operation and faults 3, 9 and 15 remain mutually confounded after the correction, consistent with their known weak observability.

Keywords: Tennessee Eastman Process, fault diagnosis, machine learning, process monitoring, reproducible benchmark, fault-onset labeling

31 1. Introduction

32 Industrial processes operate through interacting physical, chemical, and
33 control mechanisms. A local disturbance can propagate through feedback
34 loops, alter several measured variables, and eventually compromise product
35 quality, equipment integrity, safety, or environmental performance. Fault
36 detection and diagnosis (FDD) therefore constitute central elements of mod-
37 ern process monitoring [1, 2, 3]. Detection determines whether the process
38 has departed from acceptable operation, while diagnosis seeks to identify the
39 underlying condition so that operators or automated systems can respond
40 appropriately.

41 Model-based diagnosis can be effective when reliable first-principles mod-
42 els are available. However, plant-wide systems are frequently nonlinear, high-
43 dimensional, stochastic, and only partially observed. These properties have
44 motivated data-driven monitoring, in which statistical or machine-learning
45 models infer diagnostic boundaries from historical measurements [4, 2]. Clas-
46 sical multivariate methods such as principal-component analysis (PCA), par-
47 tial least squares (PLS), and Fisher discriminant analysis established impor-
48 tant foundations for process monitoring [5]. More recent studies increasingly
49 employ tree ensembles, support vector machines, neural networks, and se-
50 quence models to represent nonlinear relationships and dynamic behavior.

51 The Tennessee Eastman Process (TEP), originally introduced by Downs
52 and Vogel [6], remains one of the most widely used benchmarks in this field.
53 It represents a reactor–separator–recycle chemical process with 41 measured
54 variables, 11 manipulated variables, and a diverse set of disturbances. Its
55 continued value arises from the coexistence of relatively distinctive faults
56 and difficult conditions whose trajectories overlap normal operation. The
57 expanded simulation data published by Rieth et al. [7] provide hundreds
58 of independent runs per condition and non-overlapping random seeds for
59 development and testing, enabling large-scale and reproducible evaluation.

60 Despite extensive use of the TEP, reported results are often difficult to
61 compare. Studies may split temporally adjacent observations rather than
62 complete runs, tune models on the test set, report only accuracy, or omit
63 class-level behavior. Treating autocorrelated observations as independent
64 experimental replicates can produce optimistic estimates, while a high global
65 score can hide failures in specific operating conditions. These issues are es-
66 pecially relevant when the normal state is confused with a fault, because false
67 alarms can reduce operator confidence and generate unnecessary interven-

tions.

This study addresses these limitations through a leakage-controlled benchmark of six classical supervised classifiers: logistic regression, decision tree, random forest, histogram-based gradient boosting, linear support vector machine (SVM), and XGBoost. The study asks five questions:

1. Which classical model provides the strongest macro-averaged diagnostic performance under repeated validation?
2. Does the selected model maintain its performance on a complete, independent official test partition?
3. Which operating conditions account for the remaining diagnostic errors?
4. How much of the reported error is attributable to the fault-onset labeling convention rather than to the classifier?
5. Which model offers the best compromise between diagnostic performance and computational cost?

The main contribution is an auditable protocol that separates model selection from final testing, preserves complete simulation runs, quantifies variability over five seeds, reports confidence intervals, and examines precision, recall, F1, and confusion patterns for all 21 classes. Rather than presenting a single headline score, the analysis identifies a compact ambiguity group that defines the practical limit of the selected classifier.

A second contribution is methodological and, in our view, of wider consequence for the TEP literature. The expanded dataset injects each fault after a fixed warm-up interval, so a fraction of every faulty run is nominally normal yet carries the fault label. This fraction differs between development runs (4%) and official test runs (16.7%). We quantify how much of the reported error and of the apparent generalization gap is produced by this convention alone, and we report every headline metric under both the literal labeling and a post-onset labeling. To our knowledge this decomposition is not routinely reported, which may account for part of the dispersion among published TEP results.

2. Related Work

2.1. Data-driven process monitoring

Data-driven FDD spans latent-variable modeling, classification, clustering, change detection, and temporal modeling. The review by Venkatasubramanian et al. [1] established a broad taxonomy covering quantitative models,

104 qualitative models, and process-history-based methods. Chiang et al. [5, 4]
105 showed how PCA, discriminant PLS, and Fisher discriminant analysis could
106 support chemical-process diagnosis. Later reviews emphasized that industrial
107 data may be nonlinear, dynamic, nonstationary, multimodal, and autocorre-
108 lated [2, 3]. These properties motivate flexible nonlinear classifiers, but also
109 demand careful validation.

110 Multivariate statistical monitoring of the TEP has a long history. Raich
111 and Çinar combined statistical process monitoring with disturbance diagnosis
112 in multivariable continuous processes [20], and multiscale extensions using
113 wavelet decompositions were introduced to separate phenomena occurring
114 at different time scales [22]. A broader survey of plant-wide disturbance
115 detection is given by Thornhill and Horch [25].

116 A substantial line of work addresses the serial correlation of process data
117 directly. Dynamic PCA augments the data matrix with time-lagged variables
118 to capture time-varying structure [35]; canonical variate analysis models the
119 dynamics explicitly and was compared against PCA and DPCA on the TEP
120 simulator by Russell et al. [36], later extended with kernel density estimation
121 for nonlinear dynamics [37]. These methods are relevant here for a reason
122 beyond their performance: they treat the temporal structure of a run as
123 the object of analysis, whereas an observation-level classifier treats each row
124 independently. The labeling question examined in Section 3.2 arises precisely
125 at that boundary.

126 2.2. Classical machine learning for TEP diagnosis

127 The families evaluated here are standard: classification and regression
128 trees [8], random forests [9], gradient boosting [10] and its regularized histogram-
129 based formulation in XGBoost [11], together with multinomial logistic regres-
130 sion and margin-based linear classification [12] as controls that test whether
131 approximately linear decision surfaces suffice. All implementations in this
132 study were based on scikit-learn [13], except XGBoost, which used its refer-
133 ence implementation [11].

134 Support vector machines have been applied to the TEP in several forms.
135 Chiang et al. combined Fisher discriminant analysis with support vector
136 machines for fault diagnosis [23]; Kulkarni et al. incorporated process knowl-
137 edge into the kernel to detect TEP faults [24]; and kernel extensions of PCA
138 and independent component analysis were compared against SVM on the
139 same benchmark [26]. These studies use kernels that the present work could
140 not afford at the sample size considered here, which is the reason the linear

141 surrogate of Section 3.4 should be read as a lower bound rather than as the
142 best attainable margin classifier.

143 Deep architectures have been applied extensively to the same benchmark.
144 Chadha and Schwung compared feedforward and convolutional networks for
145 TEP fault detection [31]; hybrid designs combining support vector machines
146 with latent-variable preprocessing [34], hierarchical LSTM structures [32],
147 and metaheuristically optimized deep models [33] have all reported strong
148 results. Most directly comparable to the present study, Lomov et al. [30]
149 evaluated recurrent, attention-based and temporal convolutional architec-
150 tures on the same expanded TEP dataset used here. The present work does
151 not propose a competing architecture; it examines a property of the evalua-
152 tion protocol that applies to all of them.

153 *2.3. Evaluation protocols and leakage*

154 Concern about evaluation protocol is not specific to process monitoring.
155 In a survey across disciplines that have adopted machine learning, Kapoor
156 and Narayanan identified leakage in seventeen fields, affecting at least 294
157 publications and in several cases producing conclusions that did not survive
158 correction [29]. Their taxonomy distinguishes eight mechanisms, two of which
159 bear directly on the present benchmark: temporal leakage, where information
160 from the future informs a prediction about the past, and non-independence
161 between training and evaluation samples, where records that share a common
162 origin are distributed across partitions and treated as independent replicates.

163 Both are latent in observation-level TEP classification. Consecutive sam-
164 ples within a simulation run are strongly autocorrelated, so a split performed
165 at the observation level places near-duplicate records on both sides of the par-
166 tition. The run-level protocol of Section 3.3 is designed to avoid this. The
167 labeling question analyzed in this paper is related but distinct: it concerns
168 not how records are divided, but whether the label attached to a record
169 describes the state of the plant at that instant.

170 *2.4. Evaluation beyond accuracy*

171 Accuracy can be misleading when performance varies substantially among
172 classes. Macro-F1 gives equal importance to every class by averaging class-
173 specific F1 scores, while weighted-F1 weights them by support. Balanced
174 accuracy averages class recalls. MCC summarizes agreement between pre-
175 dictions and true labels and remains informative for multiclass problems [14].
176 Confusion matrices are indispensable because they reveal whether errors are

Table 1: Treatment of the pre-onset window in observation-level studies on the Tennessee Eastman benchmark. Three studies on the same simulator adopt three distinct conventions: relabeling the pre-onset segment as normal, retaining it under the fault label, and not addressing it.

Study	Task	Methods	Pre-onset handling	Scored as fault
Yin et al. [27]	binary detection	SVM, PLS	stated, relabeled normal	800
Sun et al. [28]	detection + identif.	Bayesian RNN	stated, retained	960
Agarwal et al. [32]	21-class diagnosis	LSTM-SAE	not mentioned	960
This work	21-class diagnosis	six classical	stated, both protocols	960 / 800

diffuse or concentrated in specific class pairs. The present study therefore uses macro-F1 as the primary model-selection criterion and treats the other metrics as complementary evidence.

2.5. Reported treatment of the fault-onset window

The analysis reported in this paper is only consequential if the treatment of the onset window varies, or goes unstated, across published work on the same benchmark. We therefore examined the data-description and experimental-protocol sections of studies for which the full text was available, recording the diagnostic task, the stated treatment of the pre-onset segment, and the number of test samples scored against the fault label. The purpose is illustrative rather than exhaustive. Three studies on the same simulator adopt three different conventions, which is sufficient to establish that the choice is not standardized; Table 1 reports them.

The question applies to both distributions of the benchmark. In the original data of Downs and Vogel as distributed by Chiang et al. [6, 4], each condition is simulated for 24 hours in training and 48 hours in testing at a three-minute interval, giving 480 and 960 samples, with the fault injected after one hour and eight hours respectively. The pre-onset fractions are therefore 4.2% and 16.7%, essentially identical to those of the expanded dataset analyzed here. The artifact is thus a property of the benchmark design rather than of any particular redistribution of it, and the results of Section 4.4 bound what can be reported on either version.

Yin et al. [27] address the window directly. Their data description states that fault information emerges eight hours after start-up, so that the first 160 test samples appear normal while the remaining 800 are genuinely faulty, and their discussion of the classification output is explicit about the consequence: the target label is -1 over the first 160 samples and $+1$ from the 161st

204 onward. The pre-onset segment is thus relabeled as normal rather than
 205 scored against the fault, and their fault-detection rate is computed over 800
 206 samples.

207 Sun et al. [28] state the same structure – the simulator runs 160 samples
 208 in the normal state before the disturbance is introduced, then continues for
 209 800 further samples – but evaluate over all 960. Awareness of the injection
 210 instant is therefore not sufficient by itself; what determines comparability is
 211 what is done with the segment.

212 Agarwal et al. [32] describe their data as 1440 samples per class, of which
 213 the first 480 are used for calibration and the remaining 960 for testing, and
 214 identify the source distribution explicitly. The instant of fault injection is
 215 not mentioned anywhere in the data description or the experimental proto-
 216 col. The consequence is not incidental to that study’s argument: its stated
 217 objective is to improve the diagnosis of the incipient faults 3, 9 and 15, to
 218 which end pseudo-random binary signals are actively injected into the plant.
 219 Under literal labeling, part of the difficulty that motivates such an interven-
 220 tion is contributed by observations in which no fault is yet present.

221 The pattern across the three is not one of carelessness, and it admits
 222 a structural explanation. In binary detection the pre-onset segment has a
 223 natural destination: the normal class already exists in the problem, so rela-
 224 beling those observations is the obvious treatment, and the false-alarm rate
 225 is in fact defined over precisely that segment. The convention is therefore
 226 built into the evaluation. In multiclass diagnosis the same move is no longer
 227 neutral, because assigning pre-onset observations of every faulty run to class
 228 0 alters the prior of the normal condition and, in the present data, would
 229 add 1.6 million observations to a class that already competes with faults 3,
 230 9 and 15. Faced with that, the simplest course is to label each run by its
 231 fault number throughout, and the distinction disappears without ever being
 232 posed as a choice. What is lost in the transition from detection to diagnosis
 233 is not information about the data but a convention for handling it.

234 The practical consequence is that reported figures are not commensurable
 235 across the three treatments. For a single frozen model the difference between
 236 literal and post-onset labeling is 0.0854 in macro-F1 and 0.1231 in MCC (Sec-
 237 tion 4.4), larger than the margins that typically separate competing methods
 238 on this benchmark. We do not claim a systematic review here; the point is
 239 that three conventions coexist, that the choice is seldom identified as such,
 240 and that it should be reported alongside the unit of partitioning.

241 **[TBD: Optional: extend Table 1 with further studies as their**

242 full texts are consulted, in particular Yin et al., Lomov et al. and
243 Chadha & Schwung. Record for each the exact sentence describing
244 the data, or its absence.]

245 3. Materials and Methods

246 3.1. Tennessee Eastman Process data

247 The TEP simulates a plant-wide reactor–separator–recycle system and
248 includes 41 process measurements (XMEAS) and 11 manipulated variables
249 (XMV), totaling 52 predictors [6]. The original problem statement specifies
250 the plant but not a control structure; the simulations distributed as bench-
251 mark data use the plant-wide control scheme of Lyman and Georgakis [21],
252 so the recorded trajectories reflect both the process dynamics and the closed
253 loops acting on them. The classification task contained 21 labels: class 0 rep-
254 resented fault-free operation and classes 1–20 represented the predefined fault
255 conditions. The CSV distribution used in this work is a format conversion
256 of the expanded TEP simulation data of Rieth et al. [7, 15].

257 The four source files contained 15,330,000 observations and 55 columns:
258 the 52 process variables plus the identifiers `faultNumber`, `simulationRun`,
259 and `sample`. Development runs contained 500 observations each. Official
260 test runs contained 960 observations each. Every class had 500 official test
261 runs, yielding

$$21 \times 500 \times 960 = 10,080,000 \quad (1)$$

262 test observations. Each class therefore contributed 480,000 observations
263 to the final test.

264 3.2. Fault-onset windows and labeling protocols

265 The expanded dataset is generated by simulating each condition for 25
266 hours (development) or 48 hours (official test) with a sampling interval of
267 three minutes, and by injecting the fault after a fixed warm-up interval: one
268 hour in faulty development runs and eight hours in faulty test runs [7, 16, 17].
269 In sample indices, the fault is active from sample 21 in development runs and
270 from sample 161 in official test runs. Fault-free runs contain no injection.

271 Consequently, the labels distributed with the dataset are only condition-
272 ally correct. In a faulty development run, samples 1–20 (4.0% of the run)
273 describe nominal operation while carrying a fault label; in a faulty official

274 test run, samples 1–160 (16.7% of the run) do so. Two consequences follow
 275 directly and are, to our knowledge, seldom made explicit.

276 First, under an observation-level protocol with literal labels, the recall of
 277 every fault class is bounded above by

$$R_c^{\max} = \frac{960 - 160}{960} = 0.8333, \quad c = 1, \dots, 20, \quad (2)$$

278 whenever the classifier assigns pre-onset observations to some class other
 279 than c . No such ceiling applies to class 0, whose test runs contain no injection.
 280 Second, because the mislabeled fraction differs between development (4.0%)
 281 and test (16.7%), a metric computed on the two partitions is not measuring
 282 the same quantity, and part of any observed validation-to-test degradation
 283 is attributable to this asymmetry rather than to generalization.

284 We therefore report all headline metrics under two labeling protocols:

- 285 • **Literal labeling (L)**: every observation retains the label distributed
 286 with the dataset. This is the protocol implicitly adopted by most pub-
 287 lished observation-level TEP results and is retained here for compara-
 288 bility.
- 289 • **Post-onset labeling (P)**: pre-onset observations of faulty runs are
 290 excluded *at evaluation time*, so that each scored observation carries a
 291 label consistent with the physical state of the plant. The model is not
 292 refitted: protocol P rescores the predictions of the same frozen model
 293 that was fitted under protocol L, including its 4% of contaminated
 294 training observations. The comparison L versus P therefore isolates
 295 the labeling convention alone, and the reported post-onset figures are
 296 conservative, since a model additionally trained without the contami-
 297 nated observations could only do better.

298 A third variant, in which pre-onset observations are excluded from fitting
 299 as well, was not evaluated here; it would confound the labeling effect with a
 300 change in the training distribution and is left for future work.

301 Protocol L is used for model selection, so that the frozen-model decision
 302 is unaffected by the present analysis. Protocol P is reported alongside it in
 303 Section 4.4. A third option, relabeling pre-onset observations as class 0, was
 304 not adopted because it alters the class prior of the normal condition and
 305 would confound the analysis of the ambiguity group.

306 3.3. Leakage-controlled partitioning and preprocessing

307 The original development runs were divided at the run level into training
 308 and validation partitions using a fixed random seed and a stratified 80/20
 309 split within each source and class. This produced 8,400 training runs (400
 310 per class), 2,100 validation runs (100 per class), and 10,500 official test runs
 311 (500 per class). No run was assigned to more than one partition.

312 All 52 predictors were standardized using

$$z_{ij} = \frac{x_{ij} - \mu_j^{(\text{train})}}{\sigma_j^{(\text{train})}}, \quad (3)$$

313 where $\mu_j^{(\text{train})}$ and $\sigma_j^{(\text{train})}$ were estimated exclusively from the 4.2 million
 314 training observations. The fitted transformation was then applied unchanged
 315 to validation and test data. This procedure prevented information from the
 316 validation or test partitions from influencing preprocessing.

317 3.4. Candidate classifiers

318 Six model families were compared. Hyperparameter tuning evaluated
 319 three candidates per family, producing 18 configurations. The selected con-
 320 figurations were subsequently frozen:

- 321 • **Logistic regression:** $C = 1.0$, maximum 1,000 iterations;
- 322 • **Decision tree:** unrestricted depth and minimum leaf size 1;
- 323 • **Random forest:** 400 trees, unrestricted depth, 50% of features con-
 324 sidered per split, and minimum leaf size 1;
- 325 • **HistGradientBoosting:** learning rate 0.05, 350 boosting iterations,
 326 and at most 31 leaf nodes;
- 327 • **Linear SVM:** stochastic-gradient hinge-loss classifier with $\alpha = 10^{-5}$,
 328 maximum 2,000 iterations, and tolerance 10^{-3} ;
- 329 • **XGBoost:** 250 trees, maximum depth 6, learning rate 0.1, row sub-
 330 sampling 0.8, column subsampling 0.8, multiclass probabilistic objec-
 331 tive, and histogram tree construction.

332 The tuning stage ranked configurations by macro-F1 and used MCC as a
 333 secondary criterion. The official test partition was not accepted as an input
 334 by either the tuning or repeated-validation programs.

335 The linear SVM was implemented as a stochastic-gradient surrogate rather
 336 than as an exact margin solver, because quadratic-programming solvers were
 337 not tractable at this sample size. Its score should therefore be read as a lower
 338 bound on linear-margin performance, and the evidence for nonlinearity re-
 339 ported in Section 5.1 rests primarily on the multinomial logistic regression,
 340 which was optimized to convergence. Kernel SVMs were not evaluated for
 341 the same computational reason.

342 3.5. Repeated validation

343 After selecting the best configuration for each model family, variability
 344 was measured across seeds 2026–2030. For each seed, 40 complete runs per
 345 class were sampled from training and independently from validation. Thus,
 346 every repetition used 420,000 training and 420,000 validation observations
 347 while retaining the full 500-observation trajectories of all selected runs.

348 For each metric, the mean, sample standard deviation, and 95% confi-
 349 dence interval were calculated across the five repetitions. With $n = 5$, the
 350 interval was

$$\bar{x} \pm t_{0.975,4} \frac{s}{\sqrt{5}}. \quad (4)$$

351 Because only five seeds were used, these intervals quantify observed seed-
 352 to-seed variability but should not be interpreted as definitive evidence of
 353 pairwise statistical superiority. In particular, overlapping intervals for the
 354 two leading models were interpreted conservatively.

355 In addition to the interval estimates, the six model families were compared
 356 with the Friedman rank test over the five seeds treated as blocks, followed
 357 by the Nemenyi post-hoc procedure [19]. Pairwise comparisons used the
 358 Wilcoxon signed-rank test with Holm correction. The critical difference of
 359 the Nemenyi procedure was computed as $CD = q_\alpha \sqrt{k(k+1)/6N}$ with $k =$
 360 6 models and $N = 5$ blocks. Results are reported in Section 4.1. With
 361 five blocks these procedures have low power by construction, and they are
 362 reported to document that the prespecified ranking rule was not substituted
 363 for an inferential check, not to establish pairwise superiority.

364 Because a single frozen evaluation cannot express sampling uncertainty,
 365 the official-test metrics were additionally accompanied by 95% intervals ob-

366 tained by resampling complete runs with replacement (1,000 bootstrap repli-
367 cates over the 10,500 test runs). The run, not the observation, is the resam-
368 pling unit, consistent with the argument that temporally adjacent observa-
369 tions are not independent replicates.

370 3.6. Frozen final evaluation

371 XGBoost candidate 1 was selected because it achieved the highest repeated-
372 validation macro-F1. The final seed was fixed at 2026 before test evaluation.
373 The final development set combined 40 complete runs per class from train-
374 ing and 40 from validation, totaling 840,000 observations. The frozen model
375 was then evaluated once on all 10,080,000 official test observations, without
376 subsampling or post-test adjustment.

377 Two earlier executions aborted during structural validation, before any
378 model fitting or metric computation, because of an incorrect run-length as-
379 sertion; only that assertion was changed, and the full incident log is available
380 in the repository.

381 The choice of 840,000 development observations, rather than the full
382 5,250,000 available, was imposed by the computational budget. To verify
383 that this choice does not materially affect the conclusions, a learning curve
384 was computed on the validation partition over $\{10, 20, 40, 80, 160\}$ complete
385 runs per class: **[TBD: learning-curve result and saturation point]**.

386 3.7. Performance measures and reproducibility

387 The reported measures were accuracy, balanced accuracy, precision, re-
388 call, class-specific F1, macro-F1, weighted-F1, MCC, confusion matrix, train-
389 ing time, inference time, inference time per observation, and serialized model
390 size. Zero divisions in class-specific measures were assigned zero. Data and
391 output integrity were monitored with SHA-256 hashes and manifests. The
392 final execution used Python 3.12.13, scikit-learn 1.9.0, and XGBoost 3.4.0 on
393 Linux.

394 4. Results

395 4.1. Model selection under repeated validation

396 Table 2 summarizes the five-seed validation results. XGBoost ranked first
397 with macro-F1 of 0.7683 ± 0.0017 and MCC of 0.7426 ± 0.0018 . HistGradi-
398 entBoosting was a close second, with macro-F1 of 0.7643 ± 0.0034 . Random
399 forest achieved 0.7398, followed by decision tree at 0.6714. The two linear

Table 2: Repeated-validation performance over five seeds. Macro-F1 was the prespecified primary selection criterion.

Rank	Model	Macro-F1 (mean $\pm$ SD)	95% CI	MCC (mean $\pm$ SD)
1	XGBoost	0.7683 ± 0.0017	[0.7661, 0.7705]	0.7426 ± 0.0018
2	HistGradientBoosting	0.7643 ± 0.0034	[0.7601, 0.7686]	0.7391 ± 0.0045
3	Random Forest	0.7398 ± 0.0021	[0.7372, 0.7424]	0.7159 ± 0.0021
4	Decision Tree	0.6714 ± 0.0013	[0.6699, 0.6730]	0.6537 ± 0.0015
5	Logistic Regression	0.4685 ± 0.0074	[0.4592, 0.4777]	0.4567 ± 0.0073
6	Linear SVM	0.4438 ± 0.0099	[0.4314, 0.4561]	0.4377 ± 0.0109

400 baselines performed substantially worse: logistic regression achieved 0.4685
 401 and linear SVM achieved 0.4438.

402 The Friedman test over the five seeds rejected the hypothesis of equal
 403 performance among the six families ($\chi^2_5 = 24.54$, $p = 1.71 \times 10^{-4}$), with a
 404 Kendall coefficient of concordance of $W = 0.982$. The ordering is thus almost
 405 perfectly consistent from seed to seed: XGBoost obtained the best macro-F1
 406 in four of the five repetitions, and the mean ranks were 1.2 (XGBoost), 1.8
 407 (HistGradientBoosting), 3.0 (random forest), 4.0 (decision tree), 5.0 (logistic
 408 regression) and 6.0 (linear SVM).

409 Post-hoc resolution is nevertheless limited. With $N = 5$ blocks the Ne-
 410 menyi critical difference is 3.372 mean ranks, so only three of the fifteen
 411 pairwise comparisons are individually significant, all of them between a tree
 412 ensemble and a linear baseline. The separation between XGBoost and Hist-
 413 GradientBoosting is 0.6 mean ranks, far below the critical difference. The
 414 Wilcoxon signed-rank test cannot help here either: with five paired observa-
 415 tions its smallest attainable two-sided p -value is 0.0625, so no comparison can
 416 reach $\alpha = 0.05$ even under a perfect win record, and after Holm correction
 417 none does.

418 These two facts are complementary rather than contradictory. The high
 419 concordance shows that the ranking is not an artifact of seed choice; the
 420 wide critical difference shows that five repetitions cannot certify small dif-
 421 ferences. Both support the conservative reading adopted here: the ordering
 422 is reproducible, the gap between the two leading models is not resolvable at
 423 this sample size, and additional seeds would be required to settle it.

424 Figure 1 shows the ranking and its uncertainty. The confidence intervals
 425 of XGBoost and HistGradientBoosting overlapped. Accordingly, XGBoost
 426 was selected by the prespecified macro-F1 rule, but the result is not presented
 427 as proof of a statistically significant difference between the two models. The

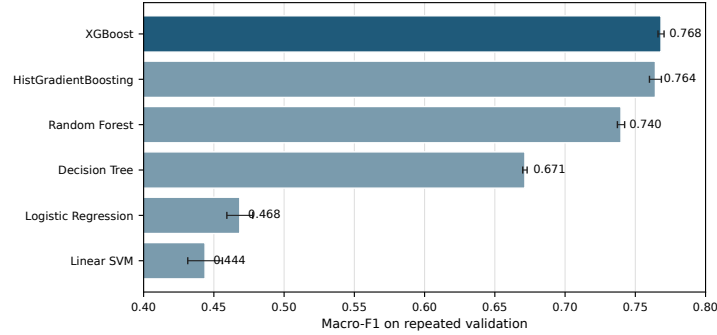


Figure 1: Repeated-validation macro-F1 for the six candidate algorithms. Error bars denote 95% confidence intervals across five seeds.

large separation between nonlinear ensembles and linear baselines indicates that nonlinear interactions are central to this diagnostic task.

4.2. Official test performance

After the protocol was frozen, the selected XGBoost model was fitted to 840,000 development observations and evaluated on the complete official test set. Table 3 presents the final metrics under literal labeling. Accuracy and balanced accuracy were both 0.6754, macro-F1 and weighted-F1 were both 0.7200, and MCC was 0.6621. The equality of accuracy and balanced accuracy, and of macro-F1 and weighted-F1, follows from the exactly balanced class supports, and holds only under literal labeling; under the post-onset protocol of Section 4.4 the fault classes retain 400,000 observations while the fault-free class retains 480,000, so the two pairs of metrics separate. Macro-F1 exceeds accuracy because accuracy equals macro-recall under balanced supports, while precision exceeds recall in most classes; the harmonic mean of a high precision and a moderate recall is larger than the recall alone. The gap between macro-F1 and accuracy is therefore itself a signature of the systematic under-recall documented in Section 4.4.

Table 3: Frozen XGBoost performance on the official test set.

Metric	Value
Accuracy	0.6754
Balanced accuracy	0.6754
Macro-F1	0.7200
Weighted-F1	0.7200
MCC	0.6621
Training time (s)	170.34
Inference time (s)	56.57
Inference (ms/observation)	0.005612
Development observations	840,000
Test observations	10,080,000

Relative to repeated validation, macro-F1 decreased by 0.0483 and MCC by 0.0805. Three mechanisms could account for this difference: independent random states in the test simulations, the longer duration of test runs, and the larger fraction of mislabeled pre-onset observations. The third mechanism is both the largest and the only one that is purely a labeling artifact. Under literal labeling the attainable recall per fault class falls from $480/500 = 0.96$ in development runs to $800/960 = 0.8333$ in test runs, a ratio of 0.868, which is of the same order as the observed degradation. Section 4.4 shows that the gap does not merely shrink under post-onset labeling but changes sign: the post-onset test macro-F1 of 0.8054 exceeds the literal-label validation figure of 0.7683 by 0.0371.

That comparison alone would not be symmetric, because the validation figure was itself computed under literal labels and carries its own 4% pre-onset contamination. The validation partition was therefore rescored under both protocols as well. Applying the selected XGBoost configuration to all 1,050,000 validation observations gave a literal-label macro-F1 of 0.7706, which agrees with the five-seed repeated-validation figure of 0.7683 to within 0.0023 and confirms that this partition is a faithful reference. Excluding the 40,000 pre-onset observations, 3.81% of the partition, raised macro-F1 to 0.7953, accuracy from 0.7556 to 0.7843, and MCC from 0.7440 to 0.7739.

The correction on the validation partition is thus +0.0246 in macro-F1, against +0.0854 on the test partition, a ratio of 0.29 that tracks the ratio of contaminated fractions (3.81% against 15.87%). Transferring the measured validation correction to the repeated-validation reference gives a post-

onset validation macro-F1 of approximately 0.7929, against a post-onset test macro-F1 of 0.8054. Table 4 summarizes the decomposition.

Table 4: Validation-to-test difference in macro-F1 under both labeling protocols. The gap present under literal labeling does not survive a symmetric comparison.

Protocol	Validation	Test	Difference
Literal (L)	0.7683	0.7200	+0.0483
Post-onset (P)	0.7929	0.8054	−0.0125

Under a symmetric comparison the gap is not merely reduced but reversed, and its magnitude falls to roughly a quarter of the literal-label value, within the range of the seed-to-seed variability of Table 2. We therefore conclude that the reported generalization gap is an artifact of the labeling convention rather than evidence of degradation on unseen trajectories, and that the literal-label figure of 0.7200 understates the discriminative capacity of the frozen model by 0.0854 in macro-F1.

Two caveats attach to this conclusion. The validation rescoring used the tuned configuration fitted to the full training partition, not the five per-seed refits of the repeated-validation procedure, so the transferred correction is an estimate of the effect rather than an exact recomputation; the close agreement between 0.7706 and 0.7683 bounds the error of that substitution. Furthermore, the pre-onset destinations differ between partitions: 61.1% of pre-onset validation observations were assigned to the group $\{0, 3, 9, 15\}$ against 92.0% in the test partition, which is consistent with the development segment being only 20 samples long and overlapping the simulation transient.

4.3. Class-specific performance

Figure 2 and Table 5 demonstrate marked heterogeneity. Seventeen classes achieved F1 scores from 0.7429 to 0.9080. Classes 6 and 7 were strongest, with F1 of 0.9080 and 0.9075, respectively. Classes 1, 2, 5, and 14 also approached or exceeded 0.8958.

Four classes were persistently difficult: class 9 (F1=0.1635), class 15 (0.1683), normal operation (class 0; 0.1718), and class 3 (0.2357). Class 3 had the highest recall within this group (0.4218) but very low precision (0.1636), indicating systematic overprediction.

A pattern in the recall column deserves emphasis. Classes 6 and 7 reached recalls of 0.8333 and 0.8332, which coincide with the structural ceiling of

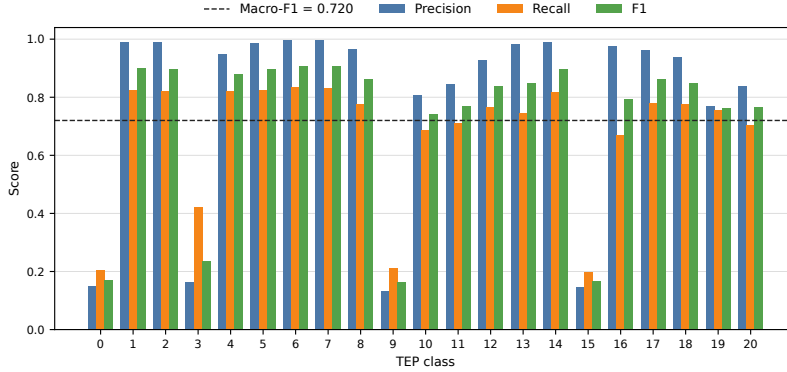


Figure 2: Class-wise precision, recall, and F1 on the complete official test partition. The dashed line indicates overall macro-F1. Class 0 is normal operation.

Eq. (2) to within one part in ten thousand. Classes 1, 2, 4, 5 and 14 lie between 0.8187 and 0.8243, that is, at 98–99% of that ceiling. The frozen model is therefore essentially saturated on these classes once the pre-onset segment is removed, and their reported F1 values of roughly 0.88–0.91 are limited by the labeling convention rather than by the classifier.

4.4. Effect of the fault-onset window

Table 6 compares the frozen model under the two labeling protocols of Section 3.2. No refitting, retuning or reselection was performed; the same frozen model and the same predictions are scored under two label conventions.

Excluding the 1,600,000 pre-onset observations, 15.87% of the partition, raised macro-F1 by 0.0854, accuracy by 0.1195 and MCC by 0.1231. No model was refitted and no hyperparameter was changed.

Two diagnostics establish that this is a labeling effect rather than a favourable choice of subset. First, the frozen model assigned 16.10% of the pre-onset observations to class 0 and 75.86% to classes 3, 9 or 15, that is, 92.0% to the group of conditions that are indistinguishable from nominal operation, and only 4.20% to the fault label they nominally carry. The model therefore identifies the pre-onset segment as a period in which nothing detectable is occurring, which is correct, and literal labeling scores this as error. Second, restricting the evaluation to post-onset observations raised the recall of the 17 well-separated classes to between 0.8015 and 0.9999, the upper value corresponding to essentially complete recovery of faults 6 and 7, while the

Table 5: Class-wise results for the frozen XGBoost model on the official test set. Every class had support of 480,000 observations.

Class	Precision	Recall	F1	Class	Precision	Recall	F1
0	0.1481	0.2045	0.1718	11	0.8447	0.7095	0.7712
1	0.9894	0.8243	0.8993	12	0.9271	0.7669	0.8395
2	0.9896	0.8209	0.8974	13	0.9828	0.7444	0.8471
3	0.1636	0.4218	0.2357	14	0.9889	0.8187	0.8958
4	0.9477	0.8222	0.8805	15	0.1465	0.1978	0.1683
5	0.9879	0.8234	0.8982	16	0.9756	0.6681	0.7931
6	0.9973	0.8333	0.9080	17	0.9624	0.7809	0.8622
7	0.9964	0.8332	0.9075	18	0.9367	0.7760	0.8488
8	0.9665	0.7779	0.8620	19	0.7701	0.7559	0.7630
9	0.1328	0.2126	0.1635	20	0.8391	0.7029	0.7650
10	0.8066	0.6885	0.7429				

Table 6: Frozen XGBoost under literal (L) and post-onset (P) labeling on the official test partition. Intervals are 95% run-level bootstrap intervals over 1,000 replicates.

Metric	Literal (L)	Post-onset (P)
Accuracy	0.6754 [0.6716, 0.6797]	0.7949 [0.7898, 0.8006]
Balanced accuracy	0.6754 [0.6748, 0.6760]	0.8006 [0.7998, 0.8013]
Macro-F1	0.7200 [0.7192, 0.7208]	0.8054 [0.8044, 0.8063]
MCC	0.6621 [0.6579, 0.6666]	0.7852 [0.7798, 0.7910]
Observations	10,080,000	8,480,000

ambiguity group improved only marginally (F1 of 0.2217, 0.3238, 0.2100 and 0.2106 for classes 0, 3, 9 and 15, against 0.1718, 0.2357, 0.1635 and 0.1683 under literal labels). The ambiguity group is therefore not a labeling artifact; the ceiling on the remaining seventeen classes is.

The two phenomena are nevertheless connected. Class 3 was predicted 1,237,562 times against a true support of 480,000, an excess of 757,562 observations. A substantial part of that excess consists of pre-onset segments belonging to other faults, which the model routes to the weakly observable conditions because, at those instants, nothing has yet happened. The over-prediction of class 3 is thus partly a consequence of the labeling convention and partly a consequence of genuine overlap, and only the post-onset analysis separates the two.

Table 7: Effect of post-onset labeling across model families. The last column reports the relative accuracy gain $\Delta A/A_P$, whose structural upper bound is $1 - 8,480,000/10,080,000 = 0.1587$.

Model	Macro-F1 (L)	Macro-F1 (P)	$\Delta A/A_P$
Decision tree	0.6330	0.7167	0.148
Extra trees	0.6965	0.7786	0.148
Random forest	0.7081	0.7912	0.149
HistGradientBoosting	0.7128	0.7972	0.149
XGBoost [†]	0.7200	0.8054	0.150
Logistic regression	0.4283	0.4698	0.142
Linear SVM	0.4207	0.4655	0.142

[†]Frozen model of Table 3, complete test partition.

533 This distinction matters for comparability. As documented in Section 2.5,
534 published observation-level results on the expanded TEP dataset do not con-
535 sistently state whether pre-onset samples were retained, discarded, or rela-
536 beled, yet the three choices are separated by several points of macro-F1 for an
537 identical model. Two studies reporting different scores may therefore differ
538 in labeling convention rather than in method. We recommend that the treat-
539 ment of the onset window be reported explicitly as part of the experimental
540 protocol, in the same way that the unit of partitioning is reported.

541 4.5. Generality of the labeling effect across model families

542 If the effect of Section 4.4 is a property of the labeling convention rather
543 than of the selected classifier, it should appear in every model evaluated on
544 the same partition. To test this, an auxiliary experiment was rescored un-
545 der both protocols. That experiment predates the frozen benchmark, uses
546 a per-class subsample of the test partition, and includes an extremely ran-
547 domized trees model in place of XGBoost; its absolute levels are therefore
548 not comparable with Table 3, and only the differences between protocols are
549 interpreted. No model was refitted.

550 The five tree-based models gained between 0.0821 and 0.0854 in macro-
551 F1, a spread of 0.003 across estimators whose capacities differ by orders of
552 magnitude, from a single unpruned tree to a regularized boosting ensemble.
553 No overfitting mechanism produces that uniformity, because the degradation
554 associated with overfitting scales with model capacity.

555 The relative accuracy gain explains the pattern quantitatively. If a clas-
 556 sifier assigns pre-onset observations to any class other than the labeled fault,
 557 its literal accuracy is its post-onset accuracy scaled by the retained fraction
 558 of the partition, so that $\Delta A/A_P$ is bounded above by 0.1587 independently
 559 of the model. The seven models fall between 0.142 and 0.150, the residual
 560 being the small fraction of pre-onset observations that match the literal label
 561 by chance. The linear baselines gain less in absolute terms, 0.0415 and
 562 0.0448, simply because they start lower; in relative terms they are affected
 563 almost identically. They also route far fewer pre-onset observations to the
 564 group $\{0, 3, 9, 15\}$, 20.6% and 41.9% against 65.7–70.2% for the ensembles,
 565 and are penalized in the same proportion regardless.

566 The consequence is that the effect is not a property of any estimator.
 567 Any observation-level result reported on this dataset under literal labels un-
 568 derstates the post-onset discriminative capacity of the model by a factor
 569 predictable from its own accuracy. Comparisons between published results
 570 are correspondingly affected whenever the treatment of the onset window
 571 differs or is unstated.

572 *4.6. Structure of the remaining errors*

573 The normalized confusion matrix in Fig. 3 shows that errors were not
 574 uniformly distributed. They concentrated in the four-class group $\{0, 3, 9, 15\}$.
 575 Among true class-9 observations, 32.39% were predicted as class 3, 20.24%
 576 as class 0, and 18.08% as class 15. Normal observations were assigned to
 577 classes 3, 9, and 15 in 32.02%, 21.51%, and 18.24% of cases, respectively. For
 578 class 15, the corresponding proportions were 30.96%, 20.53%, and 20.03%
 579 for predicted classes 3, 9, and 0.

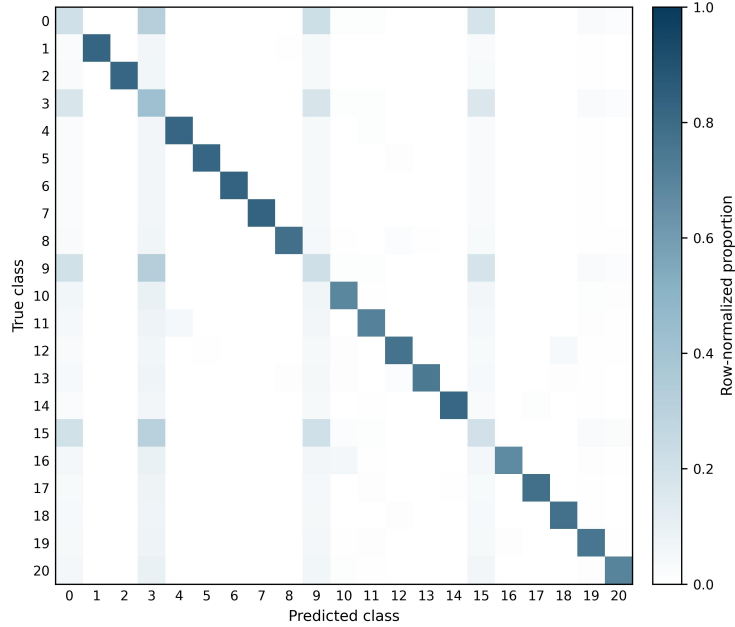


Figure 3: Row-normalized confusion matrix for the frozen XGBoost model on the official test partition.

Class 3 was predicted 1,237,562 times despite having 480,000 true observations. This excess explains its low precision and demonstrates that the model used class 3 as a frequent destination for ambiguous trajectories. The persistence of the same difficult group during repeated validation and official testing suggests an intrinsic overlap under an observation-level representation rather than an isolated sampling artifact.

4.7. Computational performance

Table 8 reports the cost of all six families under identical conditions during repeated validation, which is required to answer the performance–cost question and cannot be inferred from the selected model alone. Training times are reported as means over the five seeds and are not directly comparable with the 170.34s of Table 3, which was measured during the final evaluation on twice as many observations. The degree of parallelism was an invocation-time argument of each stage rather than a fixed setting, and it was not recorded in the run manifests, so the absolute times of the two stages reflect different thread counts on a 24-core host. The comparison between

Table 8: Performance and computational cost of the six families under repeated validation (mean over five seeds, 420,000 training observations per repetition). Times measured on the environment of Section 3.7.

Model	Macro-F1	MCC	Train (s)	Inference (μ s/obs)	Size (MB)
XGBoost	0.7683	0.7426	864.7	7.03	18.9
HistGradientBoosting	0.7643	0.7391	73.5	29.40	16.9
Random Forest	0.7398	0.7159	477.3	20.29	8703.8
Decision Tree	0.6714	0.6537	54.0	0.24	28.8
Logistic Regression	0.4685	0.4567	65.4	0.15	0.005
Linear SVM	0.4438	0.4377	4.1	0.16	0.006

families within Table 8 is unaffected, because those six runs shared a single invocation, but the cross-stage comparison is not meaningful and no conclusion is drawn from it. Recording the degree of parallelism alongside the timing is a reproducibility requirement that this benchmark did not initially meet, and it has been added to the released code.

Final XGBoost training required 170.34 seconds. Prediction over 10.08 million observations required 56.57 seconds, corresponding to 0.005612 ms per observation or approximately 178,000 observations per second. The serialized final model occupied approximately 6.8 MB. These values indicate that the selected classifier is computationally feasible for high-throughput offline analysis and potentially for online scoring, although end-to-end deployment latency must also include data acquisition, preprocessing, communication, and decision logic.

5. Discussion

5.1. Nonlinearity and model ranking

The ranking provides strong empirical evidence that the TEP classification boundary is not adequately represented by the evaluated linear models. XGBoost, HistGradientBoosting, and random forest substantially outperformed logistic regression and linear SVM even though all models received the same standardized predictors and run-level partitions. Tree ensembles can represent threshold effects and high-order interactions without requiring an explicit feature expansion, which is consistent with the nonlinear and feedback-driven nature of the TEP.

The difference between XGBoost and HistGradientBoosting was small: 0.0040 in mean macro-F1, with overlapping 95% confidence intervals. Thus,

the defensible conclusion is that XGBoost won under the prespecified ranking rule, not that it was conclusively superior under all resampling regimes. Table 8 makes the practical consequence explicit, and it does not favour a single model. HistGradientBoosting trained 11.8 times faster than XGBoost (73.5s against 864.7s) for a deficit of 0.0040 in macro-F1 with overlapping intervals, so it dominates whenever models are retrained frequently or the training budget is constrained. The ordering reverses at inference time, where XGBoost required $7.03\ \mu\text{s}$ per observation against $29.40\ \mu\text{s}$, a factor of 4.2 in favour of XGBoost. Neither model dominates the other: the choice follows from whether the deployment is retraining-bound or scoring-bound, and both conclusions are invisible if only the selected model is profiled.

Model size adds a third and largely independent constraint. The random forest required approximately 8.7 GB, some 460 times the XGBoost artifact, for 0.0285 less macro-F1. Whatever its statistical merits, it is not a candidate for embedded or edge deployment, and this would not be apparent from predictive metrics alone. The two linear baselines occupied roughly 5 kB and inferred in $0.15\ \mu\text{s}$ per observation, which quantifies precisely what is being paid for the nonlinear gain of about 0.30 in macro-F1.

5.2. *Why global performance is insufficient*

The official macro-F1 of 0.7200 summarizes useful performance, but it does not describe a uniformly reliable 21-class diagnostic system. If only macro-F1 or accuracy had been reported, the strong results for 17 classes would obscure the near-failure of normal operation and faults 3, 9, and 15. In operational terms, low precision for normal operation and extensive exchange between normal and faulty labels can increase false alarms and missed interventions.

This result illustrates why class-level precision, recall, F1, and confusion structure should accompany aggregate measures in industrial diagnosis. It also supports a distinction between *benchmark discrimination* and *deployment readiness*. The present model is a strong benchmark classifier for most faults, but additional safeguards are necessary before it could support autonomous decisions.

5.3. *Implications for method selection in industrial practice*

The results above matter for an engineer specifying a diagnostic system, and they matter in a way that is easy to miss. Benchmark figures circulate as evidence of capability: a vendor or an internal team reporting a macro-F1

657 of 0.80 on the TEP appears to offer a better method than one reporting 0.72.
658 Sections 4.4 and 4.5 show that this particular difference is fully accounted
659 for by the labeling convention, for an identical model. The question to put
660 to a reported figure is therefore not only which algorithm produced it, but
661 over which observations it was computed: whether the interval between the
662 disturbance entering the process and its becoming observable was scored as
663 faulty, as normal, or excluded.

664 The benchmark’s fixed injection instant has no counterpart in an op-
665 erating plant, but the underlying interval does. A disturbance entering a
666 unit does not appear simultaneously in the instrumentation: it propagates
667 through transport delay, is attenuated or temporarily masked by the control
668 loops acting to reject it, and becomes visible only when it exceeds the nor-
669 mal variability of the affected measurements. **[CO-AUTHORS: Give a**
670 **concrete order of magnitude from operating experience, and name**
671 **what governs it in practice – sensor placement, loop tuning, trans-**
672 **port lag, sampling and scan rate.]** During that interval a diagnostic
673 model receives data that carries no signature of the condition it is expected
674 to report. Whatever the label attached to those observations in a historical
675 dataset, the model cannot separate them, and a figure of merit computed over
676 them measures the labeling convention rather than the diagnostic system.

677 The group $\{0, 3, 9, 15\}$ makes the same point in a sharper form. These
678 conditions survive the correction of Section 4.4 because they leave no persis-
679 tent signature in the 52 measured variables, and the literature has reported
680 them as weakly detectable for two decades. For a deployment this is not
681 a modeling deficiency to be solved with a better classifier: it is a statement
682 about what the existing instrumentation can observe. **[CO-AUTHORS: From**
683 **practice: when a condition proves undiagnosable from the available**
684 **process data, what is the usual course – additional or relocated**
685 **instrumentation, a different measurement principle, or accepting**
686 **that the condition is handled by other means? What governs that**
687 **decision economically?]**

688 Finally, the cost results of Table 8 show that no model dominates. Hist-
689 GradientBoosting trained 11.8 times faster than XGBoost for a difference of
690 0.0040 in macro-F1, while XGBoost inferred 4.2 times faster; the random for-
691 est required approximately 8.7 GB. Which of these constraints binds depends
692 on the deployment rather than on the benchmark. **[CO-AUTHORS: From**
693 **experience: in the installations you work with, what is the actual**
694 **limiting factor – how often models are retrained, the latency budget**

695 within the scan cycle, or the memory and storage available on the
696 controller, edge device or historian? An example of a constraint
697 that ruled out an otherwise adequate method would be valuable
698 here.]

699 The common thread is that benchmark performance and deployment
700 readiness are distinct properties, and that the distance between them is not
701 only a matter of model quality. It also depends on how the evaluation was
702 constructed, on what the instrumentation can observe, and on the resources
703 available where the model will actually run.

704 5.4. Interpretation of the ambiguity group

705 The identity of the difficult group is not a new finding, and it would be
706 misleading to present it as one. Faults 3, 9 and 15 have been reported as
707 weakly detectable since the earliest systematic studies of the TEP: moni-
708 toring statistics based on PCA, discriminant PLS and Fisher discriminant
709 analysis report missed-detection rates close to those of nominal operation
710 for exactly these conditions [5, 4], and later comparative studies reproduce
711 the pattern across a wide range of data-driven statistics [18]. The contribu-
712 tion here is the scale and the protocol: the same group is recovered under
713 run-level partitioning, on 10.08 million independent test observations, and it
714 persists after the labeling artifact of Section 4.4 is removed. This strengthens
715 the interpretation that the overlap is a property of the measured variables
716 rather than of any particular estimator.

717 The mechanism, however, constrains which remedies can work. If these
718 disturbances leave no persistent signature in the 52 measured variables, then
719 temporal aggregation over a run reduces the variance of an estimate that is
720 already centered on an ambiguous region: it will sharpen the decision without
721 moving it toward the correct class. Run-level aggregation should therefore
722 be expected to help the 17 separable classes, whose pre-onset transients are
723 transient, far more than the group $\{0, 3, 9, 15\}$, and this asymmetry is itself a
724 testable prediction. A hierarchical design that isolates the ambiguous group
725 and applies a discriminator with explicit memory to classes 0, 3, 9 and 15
726 is the more promising direction, but its plausibility rests on whether any
727 signature exists at all; establishing this is a question of observability, not
728 of classifier capacity, and it is better addressed with residual-based or first-
729 principles analysis than with additional supervised tuning.

730 Both proposals arise from test-set diagnosis and must therefore be eval-
731 uated under a newly defined development/test protocol; the present official

732 test must not be reused for tuning them.

733 5.5. *Threats to validity*

734 Several limitations delimit the conclusions. First, TEP is a high-fidelity
735 simulation benchmark, not a physical plant. Results may not transfer directly
736 to sensor drift, maintenance changes, unmodeled disturbances, communica-
737 tion losses, or evolving operating modes in real facilities. Second, the bench-
738 mark evaluated pointwise multiclass diagnosis and did not measure detection
739 delay, time-to-alarm, false alarms per run, or sustained alarm logic. Third,
740 five seeds provide a useful but small basis for uncertainty estimation. The
741 resulting t intervals characterize the implemented experiment rather than all
742 possible data partitions.

743 Fourth, hyperparameter search was intentionally limited to three can-
744 didates per model family to preserve computational feasibility and trans-
745 parency. A larger search might improve individual algorithms, particularly
746 the linear baselines or random forest. Fifth, the final model was fitted to
747 840,000 of the 5,250,000 available development observations, and model se-
748 lection was carried out at 420,000; a ranking established at one training scale
749 need not be invariant to scale, and the learning curve reported in Section 4.1
750 bounds but does not eliminate this concern. Sixth, the linear SVM was a
751 stochastic-gradient surrogate and its result should not be read as the best
752 attainable linear-margin performance. Seventh, the degree of parallelism was
753 not recorded in the run manifests, so the training times of Table 8 support
754 ordinal comparison between families, which shared one invocation, but not
755 comparison against the timings of other stages. Eighth, interpretation fo-
756 cused on output behavior and did not include feature attribution or causal
757 process analysis. Finally, the dataset had balanced class support, whereas
758 real industrial faults are rare and highly imbalanced. Metrics, thresholds,
759 and alarm costs would require recalibration under field prevalence.

760 6. Conclusion

761 This work presented a reproducible classical-machine-learning benchmark
762 for 21-class fault diagnosis in the Tennessee Eastman Process. Run-level
763 splitting, train-only standardization, separate tuning and validation, five-
764 seed repetition, and a single frozen official test evaluation were used to reduce
765 leakage and post-selection bias.

766 XGBoost achieved the highest repeated-validation macro-F1 ($0.7683 \pm$
 767 0.0017) and produced an official test macro-F1 of 0.7200, accuracy of 0.6754,
 768 and MCC of 0.6621 over 10.08 million observations under literal labeling,
 769 rising to a macro-F1 of 0.8054, an accuracy of 0.7949 and an MCC of 0.7852
 770 once the pre-onset segment of each faulty run is excluded, with no refitting of
 771 any kind. The validation partition, rescored under the same protocol, moved
 772 from 0.7706 to 0.7953, so that the apparent generalization gap of +0.0483 in
 773 macro-F1 becomes -0.0125 once the comparison is made symmetric. Seven-
 774 teen classes were diagnosed with F1 between 0.7429 and 0.9080, and several
 775 of them were shown to sit at the structural recall ceiling imposed by the la-
 776 beling convention rather than at the limit of the classifier. Normal operation
 777 and faults 3, 9, and 15 remained strongly confounded, in agreement with
 778 two decades of TEP monitoring literature, and this confounding survives the
 779 removal of the labeling artifact.

780 Three conclusions follow. Nonlinear tree ensembles provide effective di-
 781 agnosis for most TEP conditions. Aggregate scores overstate practical reli-
 782 ability when a small group of classes dominates the remaining errors. And
 783 a benchmark result on the expanded TEP dataset is not interpretable, nor
 784 comparable across studies, unless the treatment of the fault-onset window
 785 is stated explicitly. A single frozen model differs by 0.085 in macro-F1 and
 786 0.123 in MCC between two defensible labeling conventions, which is larger
 787 than most of the improvements reported between competing architectures on
 788 this benchmark. The effect is moreover common to every model family tested,
 789 with a relative accuracy gain between 0.142 and 0.150 across seven classifiers,
 790 so it displaces published results without reordering them only when all stud-
 791 ies happen to adopt the same convention. We therefore recommend that the
 792 treatment of the onset window be stated as part of the experimental protocol
 793 in any observation-level study using these data.

794 Future work should investigate temporal aggregation, hierarchical diag-
 795 nosis, probability calibration, detection delay, and cost-sensitive alarm rules
 796 under a new untouched test protocol. External validation with physical in-
 797 dustrial data will be necessary before deployment-oriented conclusions can
 798 be drawn.

799 Data and Code Availability

800 The expanded Tennessee Eastman simulation data are publicly available
 801 from Harvard Dataverse [7]. The CSV conversion used in this study is avail-

able through Kaggle [15]. All code, configuration files, run-level split manifests, frozen hyperparameters, five-seed validation outputs, final predictions, class-level metrics, confusion matrices, SHA-256 data hashes, and the environment specification are deposited at an anonymized repository for peer review, **[TBD: anonymous DOI]**; the non-anonymized repository is cited in the accepted version. The analysis of the fault-onset window can be reproduced from the deposited predictions without refitting any model.

CRediT Authorship Contribution Statement

First author: Conceptualization, Investigation, Validation, Writing – original draft, Writing – review & editing. **Second author:** Investigation, Validation, Writing – original draft, Writing – review & editing. **Third author:** Conceptualization, Methodology, Software, Formal analysis, Data curation, Writing – original draft, Visualization, Supervision, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] V. Venkatasubramanian, R. Rengaswamy, K. Yin, and S. N. Kavuri, “A review of process fault detection and diagnosis: Part I: Quantitative model-based methods,” *Computers & Chemical Engineering*, vol. 27, no. 3, pp. 293–311, 2003. doi: 10.1016/S0098-1354(02)00160-6.
- [2] S. Yin, S. X. Ding, X. Xie, and H. Luo, “A review on basic data-driven approaches for industrial process monitoring,” *IEEE Transactions on Industrial Electronics*, vol. 61, no. 11, pp. 6418–6428, 2014. doi: 10.1109/TIE.2014.2301773.
- [3] C. Ji and W. Sun, “A review on data-driven process monitoring methods: Characterization and mining of industrial data,” *Processes*, vol. 10, no. 2, art. 335, 2022. doi: 10.3390/pr10020335.

- 832 [4] L. H. Chiang, E. L. Russell, and R. D. Braatz, *Fault Detection and*
833 *Diagnosis in Industrial Systems*. London, UK: Springer, 2001. doi:
834 10.1007/978-1-4471-0347-9.
- 835 [5] L. H. Chiang, E. L. Russell, and R. D. Braatz, “Fault diagnosis in chem-
836 ical processes using Fisher discriminant analysis, discriminant partial
837 least squares, and principal component analysis,” *Chemometrics and*
838 *Intelligent Laboratory Systems*, vol. 50, no. 2, pp. 243–252, 2000. doi:
839 10.1016/S0169-7439(99)00061-1.
- 840 [6] J. J. Downs and E. F. Vogel, “A plant-wide industrial process control
841 problem,” *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–
842 255, 1993. doi: 10.1016/0098-1354(93)80018-I.
- 843 [7] C. A. Rieth, B. D. Amsel, R. Tran, and M. B. Cook, “Additional Ten-
844 nessee Eastman Process simulation data for anomaly detection evalua-
845 tion,” Harvard Dataverse, V1, 2017. doi: 10.7910/DVN/6C3JR1.
- 846 [8] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification*
847 *and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
- 848 [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32,
849 2001. doi: 10.1023/A:1010933404324.
- 850 [10] J. H. Friedman, “Greedy function approximation: A gradient boosting
851 machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
852 doi: 10.1214/aos/1013203451.
- 853 [11] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting sys-
854 tem,” in *Proceedings of the 22nd ACM SIGKDD International Confer-*
855 *ence on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi:
856 10.1145/2939672.2939785.
- 857 [12] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*,
858 vol. 20, pp. 273–297, 1995. doi: 10.1007/BF00994018.
- 859 [13] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal*
860 *of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- 861 [14] B. W. Matthews, “Comparison of the predicted and observed secondary
862 structure of T4 phage lysozyme,” *Biochimica et Biophysica Acta*, vol.
863 405, no. 2, pp. 442–451, 1975. doi: 10.1016/0005-2795(75)90109-9.

- [15] A. Melo, “Tennessee Eastman Process CSV: Process simulation data for anomaly detection evaluation,” Kaggle Dataset, accessed Aug. 2026. [Online]. Available: <https://www.kaggle.com/datasets/afrniomelo/tep-csv>.
- [16] C. A. Rieth, B. D. Amsel, R. Tran, and M. B. Cook, “Issues and advances in anomaly detection evaluation for joint human-automated systems,” in *Advances in Human Factors in Robots and Unmanned Systems* (AHFE 2017), Cham: Springer, 2018, pp. 52–63.
- [17] C. Reinartz, M. Kulahci, and O. Ravn, “An extended Tennessee Eastman simulation dataset for fault-detection and decision support systems,” *Computers & Chemical Engineering*, vol. 149, art. 107281, 2021. doi: 10.1016/j.compchemeng.2021.107281.
- [18] S. Yin, S. X. Ding, A. Haghani, H. Hao, and P. Zhang, “A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark Tennessee Eastman process,” *Journal of Process Control*, vol. 22, no. 9, pp. 1567–1581, 2012. doi: 10.1016/j.jprocont.2012.06.009.
- [19] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [20] A. Raich and A. Çinar, “Statistical process monitoring and disturbance diagnosis in multivariable continuous processes,” *AIChE Journal*, vol. 42, no. 4, pp. 995–1009, 1996.
- [21] P. R. Lyman and C. Georgakis, “Plant-wide control of the Tennessee Eastman problem,” *Computers & Chemical Engineering*, vol. 19, no. 3, pp. 321–331, 1995.
- [22] M. Misra, H. H. Yue, S. J. Qin, and C. Ling, “Multivariate process monitoring and fault diagnosis by multi-scale PCA,” *Computers & Chemical Engineering*, vol. 26, no. 9, pp. 1281–1293, 2002.
- [23] L. H. Chiang, M. E. Kotanchek, and A. K. Kordon, “Fault diagnosis based on Fisher discriminant analysis and support vector machines,” *Computers & Chemical Engineering*, vol. 28, no. 8, pp. 1389–1401, 2003.

- 895 [24] A. Kulkarni, V. K. Jayaraman, and B. D. Kulkarni, "Knowledge incor-
896 porated support vector machines to detect faults in Tennessee Eastman
897 process," *Computers & Chemical Engineering*, vol. 29, no. 10, pp. 2128–
898 2133, 2005.
- 899 [25] N. F. Thornhill and A. Horch, "Advances and new directions in plant-
900 wide disturbance detection and diagnosis," *Control Engineering Prac-*
901 *tice*, vol. 15, no. 10, pp. 1196–1206, 2007.
- 902 [26] Y. W. Zhang, "Enhanced statistical analysis of nonlinear process using
903 KPCA, KICA and SVM," *Chemical Engineering Science*, vol. 64, no. 5,
904 pp. 801–811, 2009.
- 905 [27] S. Yin, X. Gao, H. R. Karimi, and X. Zhu, "Study on support vec-
906 tor machine-based fault detection in Tennessee Eastman process," *Ab-*
907 *stract and Applied Analysis*, vol. 2014, art. 836895, 8 pages, 2014. doi:
908 10.1155/2014/836895.
- 909 [28] W. Sun, A. R. C. Paiva, P. Xu, A. Sundaram, and R. D. Braatz, "Fault
910 detection and identification using Bayesian recurrent neural networks,"
911 *Computers & Chemical Engineering*, vol. 141, art. 106991, 2020. doi:
912 10.1016/j.compchemeng.2020.106991.
- 913 [29] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis
914 in machine-learning-based science," *Patterns*, vol. 4, no. 9, art. 100804,
915 2023. doi: 10.1016/j.patter.2023.100804.
- 916 [30] I. Lomov, M. Lyubimov, I. Makarov, and L. E. Zhukov, "Fault detection
917 in Tennessee Eastman process with temporal deep learning models,"
918 *Journal of Industrial Information Integration*, vol. 23, art. 100216, 2021.
919 doi: 10.1016/j.jii.2021.100216.
- 920 [31] G. S. Chadha and A. Schwung, "Comparison of deep neural network ar-
921 chitectures for fault detection in Tennessee Eastman process," in *Proc.*
922 *22nd IEEE International Conference on Emerging Technologies and*
923 *Factory Automation (ETFA)*, 2017, pp. 1–8.
- 924 [32] P. Agarwal et al., "Hierarchical deep LSTM for fault detection and diag-
925 nosis for a chemical process," *Processes*, vol. 10, no. 12, art. 2557, 2022.
926 doi: 10.3390/pr10122557.

- 927 [33] C. Anitha et al., “Fault diagnosis of Tennessee Eastman process with
928 detection quality using IMVOA with hybrid DL technique in IIoT,” *SN*
929 *Computer Science*, vol. 4, no. 5, art. 458, 2023. doi: 10.1007/s42979-
930 023-01851-9.
- 931 [34] X. Gao and J. Hou, “An improved SVM integrated GS-PCA fault di-
932 agnosis approach of Tennessee Eastman process,” *Neurocomputing*, vol.
933 174, pp. 906–911, 2016.
- 934 [35] W. Ku, R. H. Storer, and C. Georgakis, “Disturbance detection and
935 isolation by dynamic principal component analysis,” *Chemometrics and*
936 *Intelligent Laboratory Systems*, vol. 30, no. 1, pp. 179–196, 1995.
- 937 [36] E. L. Russell, L. H. Chiang, and R. D. Braatz, “Fault detection in in-
938 dustrial processes using canonical variate analysis and dynamic principal
939 component analysis,” *Chemometrics and Intelligent Laboratory Systems*,
940 vol. 51, no. 1, pp. 81–93, 2000.
- 941 [37] P. P. Odiowei and Y. Cao, “Nonlinear dynamic process monitoring using
942 canonical variate analysis and kernel density estimations,” *IEEE Trans-*
943 *actions on Industrial Informatics*, vol. 6, no. 1, pp. 36–45, 2010.